



Creating Your Own Server

The Socket API

Part—1

In this series of articles aimed at newbies to network programming (knowledge of C is a prerequisite), we'll learn how to create network clients and servers using the UNIX Socket API. We'll start with creating simple server-client programs, and later on try something more complex. We'll also try to understand how different servers work. I've tried to include a lot of details, but if you find some information missing, do mail me.

Since we're discussing sockets in network programming, newbies should first understand the OSI model layers, and the protocols used in these layers. Every layer in the model is responsible for doing some particular work, to make data transfer possible on the network. Each layer abstracts the lower layers' work to the layer above it. I recommend that if you don't know the ISO OSI (Open Systems Interconnection) Reference Model, you should read up on it. A good starting point is Wikipedia (en.wikipedia.org/wiki/OSI_model).

Here, we focus on the session layer (which establishes and maintains sessions) and the transport layer, which provides end-to-end reliable and unreliable delivery. Some protocols are TCP (for reliable connections), UDP (non-reliable connections) and SCTP (an advanced protocol with multiple-connection capability). Please also look at [www.wikipedia.org/wiki/TCP/IP_protocol_suite](http://en.wikipedia.org/wiki/TCP/IP_protocol_suite) and www.tcpipguide.com/free/t_TCPReliabilityandFlowControlFeaturesandProtocolMod.htm for information on TCP/IP.

Transmission Control Protocol (TCP)

TCP is a connection-oriented protocol that provides a reliable, full-duplex byte stream to users. Here, we will directly communicate with the transport layer, using TCP from the application layer, where users can communicate with the program.

Here are some important features of TCP.

It is a reliable protocol (the opposite of connection-

less UDP, which we will look at in later articles). After transmitting a packet, it awaits an acknowledgement; if one is not returned, it retransmits the packet a number of times (depending upon the implementation). If the data cannot be transmitted, it notifies the user and closes the connection.

TCP decides how long to wait for an acknowledgement by estimating the RTT (Round Trip Time) between server and client. It also associates sequence numbers with segments, so that segments arriving out of order can be reordered at the receiver. Duplicate segments (retransmitted due to delays) can thus also be ignored.

TCP provides flow control; the receiver can tell the sender how many bytes it is willing to receive, so that a slow receiver is not flooded.

TCP connections are full-duplex—the application can both send and receive data simultaneously

Simple servers

Now let's create a simple server to understand Internet sockets (*server.c*, which you can find on the LFY website at http://linuxforu.com/article_source_code/aug11/server.zip). Initially, our code will be for IPv4, but in later articles, we'll look at IPv6, and then move to protocol-independent code.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
```